SENG 319: Software Design Patterns

Group Project 1: Detection and Refactoring of Anti-Patterns and Code Smells
Group Name: Group_Name (Temp) | Target Codebase: CheckStyle 14.1.0

Part I: Software Design Anti-Patterns

1. Anti-Pattern Types (Design Smells) Detected by PTIDEJ

Using the PTIDEJ analysis tool on the CheckStyle codebase (outputs stored in lesson-vm-out/design smells), four Software Design Anti-Pattern detectors were evaluated. The table below lists each anti-pattern type and the number of detected instances:

Anti-Pattern Type (Design Smell)	Instances	Detected Locations / Summary
AntiSingleton	3	JavaLanguageParser (fileCounter), JavaLanguageLexer (modeNames, channelNames), JavadocCommentsLexer (modeNames, channelNames)
Blob (God Class)	7	Main, Checker, JavadocDetailNodeParser, FallThroughCheck, HandlerFactory, AbstractClassCouplingCheck, FinalLocalVariableCheck
FunctionalDecomposition	0	No instances detected
SwissArmyKnife	0	No instances detected
Total Anti-Patterns	**10**	**Across 4 Software Design Anti-Pattern categories**

2. Most Impactful Anti-Pattern on CheckStyle Quality & Justification

From the PTIDEJ detection results in (1), both Blob (7 instances) and AntiSingleton (3 instances) exert major impacts on the quality, maintainability, performance, and reliability of the CheckStyle application:

- [Blob (God Class) - Architectural & Maintainability Impact]: Blob classes (such as com.puppycrawl.tools.checkstyle.Main with McCabe complexity = 152.0 and NADE = 39.0, and Checker with McCabe complexity = 109.0) centralize an excessive number of responsibilities, fields, and methods while surrounding collaborator classes act as simple data containers. In Main.java, CLI option parsing, configuration loading, listener output stream management, file traversal, and exit-code calculation are tightly coupled into a single class. This wastes lines of code (LOC), increases cognitive load, hurts modularity, and makes independent unit testing difficult.
- [AntiSingleton - Concurrency, State Safety & Runtime Impact]: The AntiSingleton anti-pattern occurs when a class exposes non-final static fields (flagged by PTIDEJ as NotClassGlobalVariable) that function as unprotected global mutable state. In JavaLanguageParser (generated from JavaLanguageParser.g4), the package-private static field "static int fileCounter = 0;" is mutated inside the parser constructor every time CheckStyle parses a Java file to decide when to clear the ANTLR DFA state cache (_interp.clearDFA()). Because this mutable static primitive is neither private nor synchronized, multi-threaded file checking triggers race conditions (lost increments or concurrent resets), leading to non-deterministic DFA cache clearing and memory/performance degradation.

3. Refactoring Two Anti-Pattern Instances

Instance 1: AntiSingleton in com.puppycrawl.tools.checkstyle.grammar.java.JavaLanguageParser

a. Reason for Choosing the Anti-Pattern Instance:

We chose the AntiSingleton instance in JavaLanguageParser because it sits directly on CheckStyle's hot path (instantiated once per parsed Java source file) and resolves both an object-oriented encapsulation

violation and a real thread-safety race condition with a clean, self-contained in-place refactoring. Note that the other two AntiSingleton instances reported in AntiSingleton.ini (JavaLanguageLexer and JavadocCommentsLexer) stem from ANTLR 4's built-in Java code generator template (Java.stg), which emits "public static String[] channelNames" and "public static String[] modeNames" without the "final" modifier into target/generated-sources/antlr. In contrast, JavaLanguageParser.fileCounter is hand-written code inside JavaLanguageParser.g4.

b. Location Within the CheckStyle Codebase [Original Code Snippet]:

File: src/main/resources/com/puppycrawl/tools/checkstyle/grammar/java/JavaLanguageParser.g4 (lines 54-78):

```
    static int fileCounter = 0;

    public JavaLanguageParser(TokenStream input, int clearDfaLimit) {
        super(input);
        _interp = new ParserATNSimulator(this, _ATN , _decisionToDFA, _sharedContextCache);
        fileCounter++;
        if (fileCounter > clearDfaLimit) {
            _interp.clearDFA();
            fileCounter = 0;
        }
    }
```

c. Steps to Refactor and Solve the Issue & Justification:

- Step 1 (Encapsulate Mutable Counter in DfaClearCounter): Inside @parser::members of JavaLanguageParser.g4, we defined the static final helper class DfaClearCounter that encapsulates "private final int limit" and the mutable instance field "private int count".
- Step 2 (Replace Mutable Static Field with Private Static Final Constant): We removed "static int fileCounter = 0;" and replaced it in place with "private static final DfaClearCounter DFA_CLEAR_COUNTER = new DfaClearCounter(CLEAR_DFA_LIMIT);". Because the static reference is private and final, JavaLanguageParser no longer exposes a mutable static global variable (NotClassGlobalVariable).
- Step 3 (Synchronize State Transitions): We implemented synchronized tick() and tick(int customLimit) methods on DfaClearCounter so that incrementing count, comparing against the DFA limit, and resetting count to 0 happen atomically in a single critical section, eliminating data races during concurrent file parsing.

d. Refactored Code Snippet & PTIDEJ Verification Result:

```
    private static final DfaClearCounter DFA_CLEAR_COUNTER =
            new DfaClearCounter(CLEAR_DFA_LIMIT); // Anti-Pattern Refactor

    public static final class DfaClearCounter { // Anti-Pattern Refactor
        private final int limit; // Anti-Pattern Refactor
        private int count; // Anti-Pattern Refactor

        public DfaClearCounter(int limit) { // Anti-Pattern Refactor
            this.limit = limit; // Anti-Pattern Refactor
        }

        public synchronized boolean tick() { // Anti-Pattern Refactor
            return tick(limit); // Anti-Pattern Refactor
        }

        public synchronized boolean tick(int customLimit) { // Anti-Pattern Refactor
            final boolean shouldClear; // Anti-Pattern Refactor
            if (++count > customLimit) { // Anti-Pattern Refactor
                count = 0; // Anti-Pattern Refactor
                shouldClear = true; // Anti-Pattern Refactor
            }
            else {
                shouldClear = false; // Anti-Pattern Refactor
            }
            return shouldClear; // Anti-Pattern Refactor
        }
    }

    public JavaLanguageParser(TokenStream input, int clearDfaLimit) {
        super(input);
        _interp = new ParserATNSimulator(this, _ATN , _decisionToDFA, _sharedContextCache);
        if (DFA_CLEAR_COUNTER.tick(clearDfaLimit)) { // Anti-Pattern Refactor
            _interp.clearDFA(); // Anti-Pattern Refactor
        }
    }
```

PTIDEJ Resolution Result: Because "static int fileCounter" has been removed and replaced with a "private static final" reference whose mutable state is an encapsulated private instance variable inside DfaClearCounter, PTIDEJ's NotClassGlobalVariable check no longer triggers on JavaLanguageParser.

Instance 2: Blob Anti-Pattern Refactoring

[To be completed in Task 2: Blob Anti-Pattern Refactoring.]

Part II: Code Smells

1. Code Smells Detected by PTIDEJ

The table below lists all Code Smell types analysed by PTIDEJ in lesson-vm-out/design smells and their instance counts:

Code Smell Type	Instances	Notes
RefusedParentBequest	40	Subclasses overriding/ignoring parent bequest
ComplexClass	31	Classes with high McCabe cyclomatic complexity
LongParameterList	19	Methods with excessive parameter counts
LazyClass	13	Classes with very few methods/instructions (IR)
ClassDataShouldBePrivate	7	Classes exposing non-private fields

BaseClassShouldBeAbstract	2	Base classes not declared abstract
SpeculativeGenerality	2	Abstract classes with unused generality
BaseClassKnowsDerivedClass	0	None detected
LargeClass	0	None detected
ManyFieldAttributesButNotComplex	0	None detected
MessageChains	0	None detected
TraditionBreaker	0	None detected
Total Code Smells	**114**	**Across 12 Code Smell categories**

[Part II Questions 2 & 3 will be populated as Code Smell refactorings are completed.]